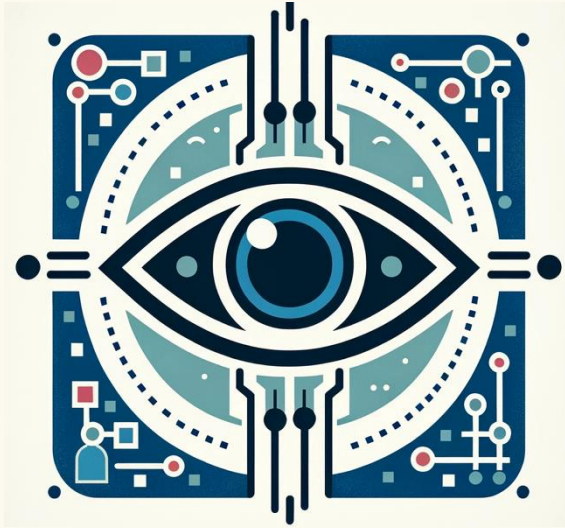




COMPUTER VISION

CNN Architectures

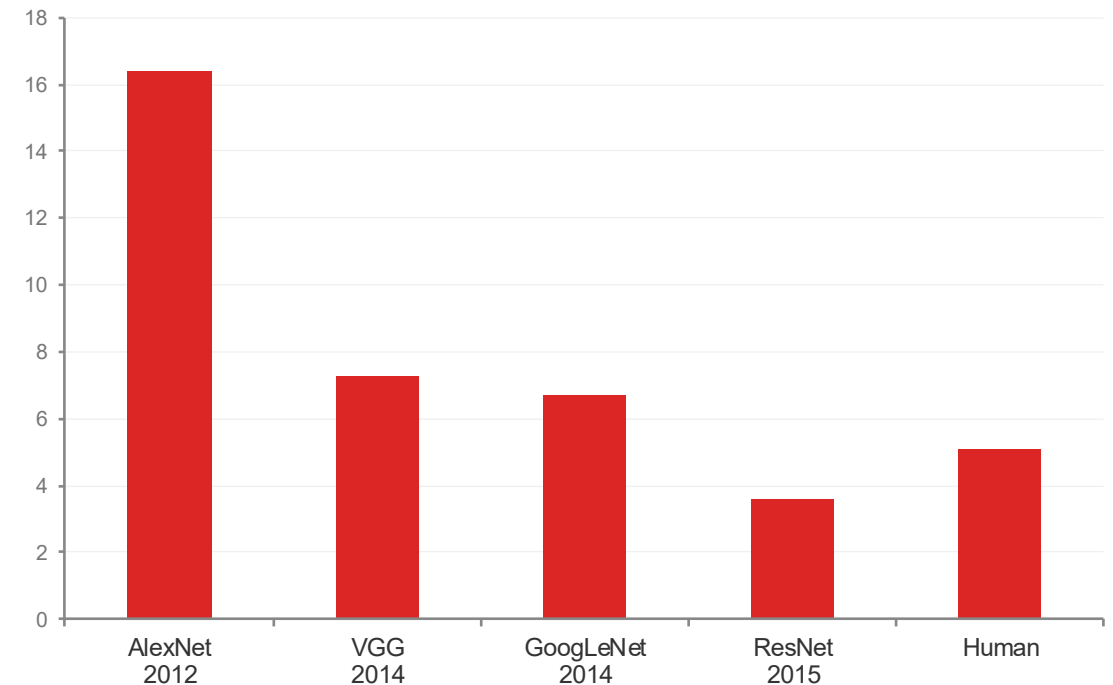
CNN Architectures

What defines a CNN architecture?

● Kernel size	$11 \times 11 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
● Depth	5 layers $\rightarrow$ 152+ layers
● Width	64 $\rightarrow$ 512+ channels per layer
● Connectivity	sequential $\rightarrow$ skip $\rightarrow$ dense
● Pooling & reduction	avg/max pool $\rightarrow$ global avg pool $\rightarrow$ strided conv

Each architecture makes different trade-offs across these dimensions.

The ImageNet classification race



In just 3 years, CNNs surpassed human-level performance on ImageNet.

AlexNet

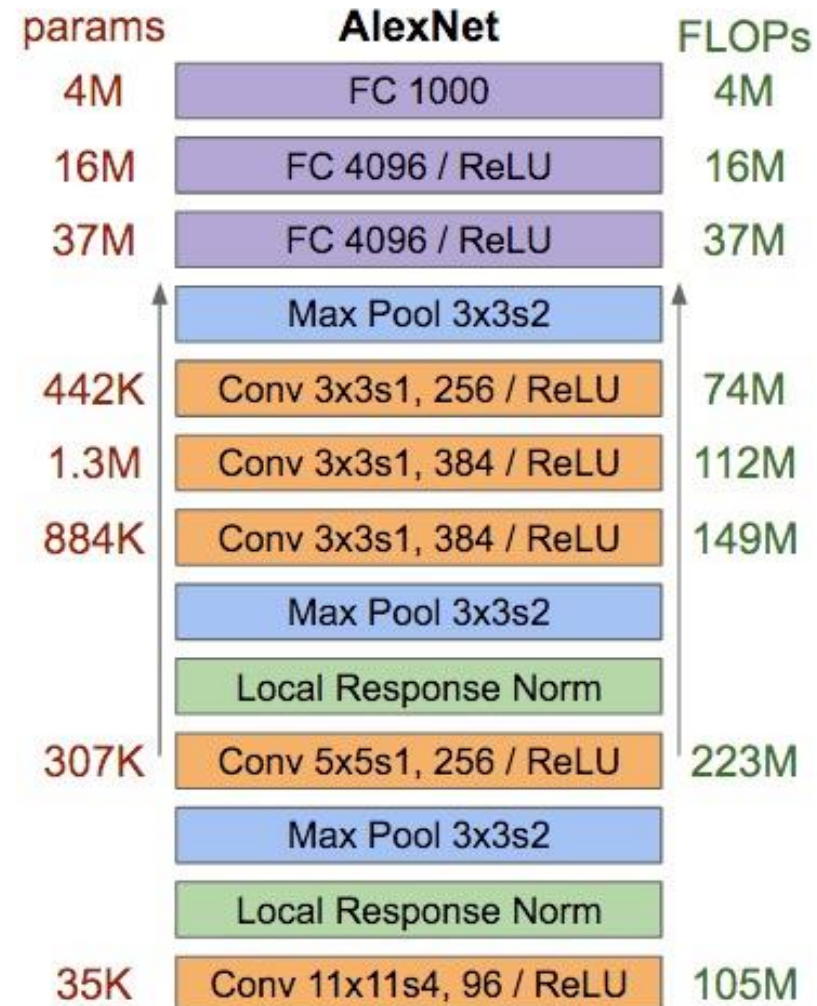
Layers defined by:

- Kernel sizes
- Strides
- # channels
- # kernels
- Pooling

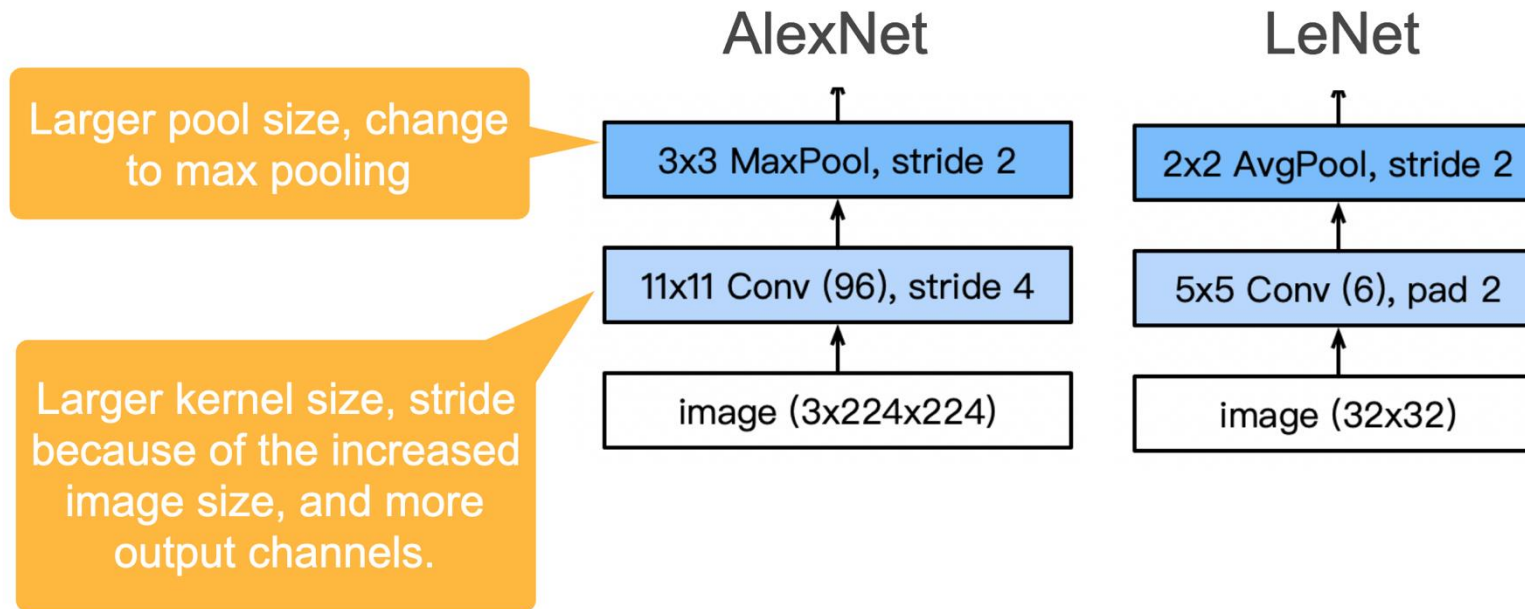
Total parameters: 61M

vs. LeNet

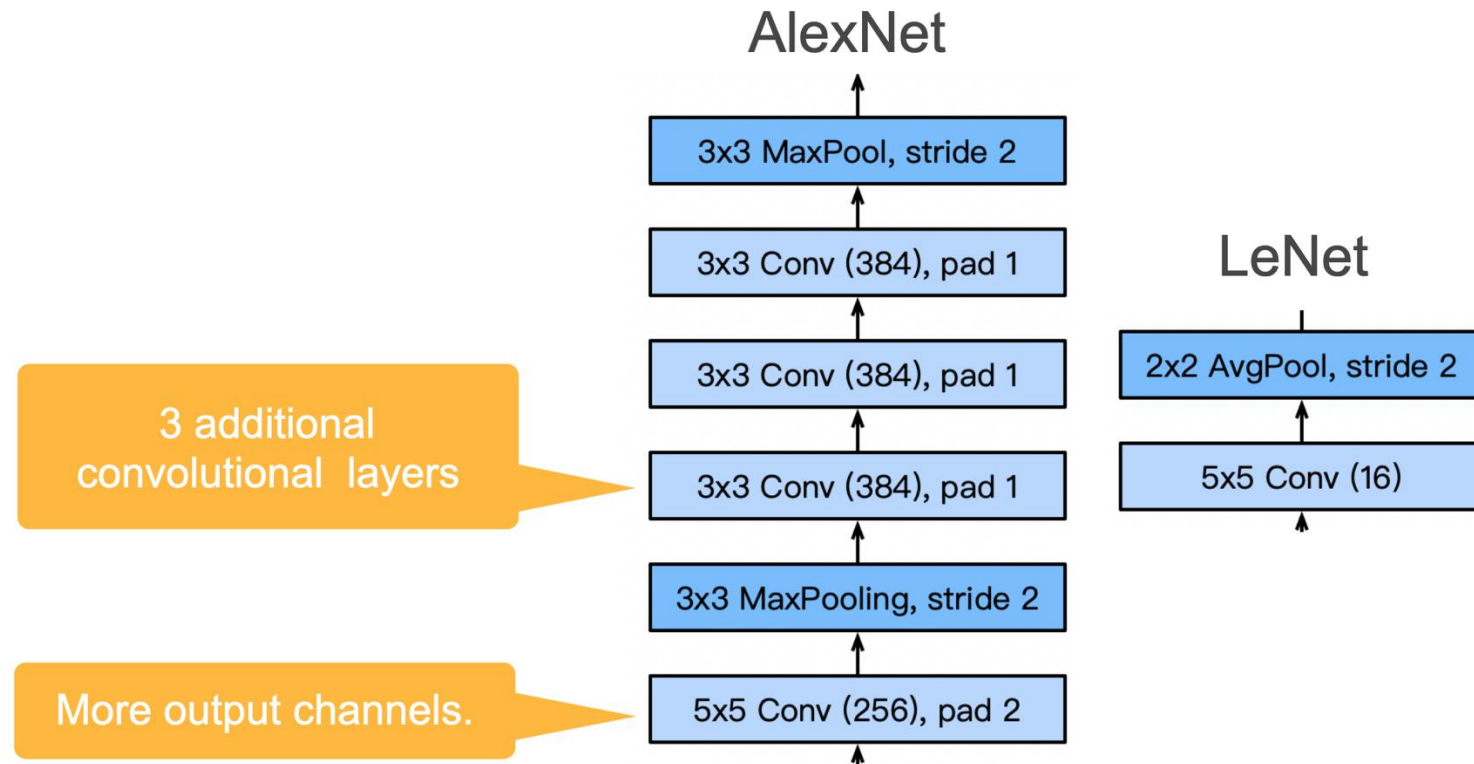
Total parameters: 60k



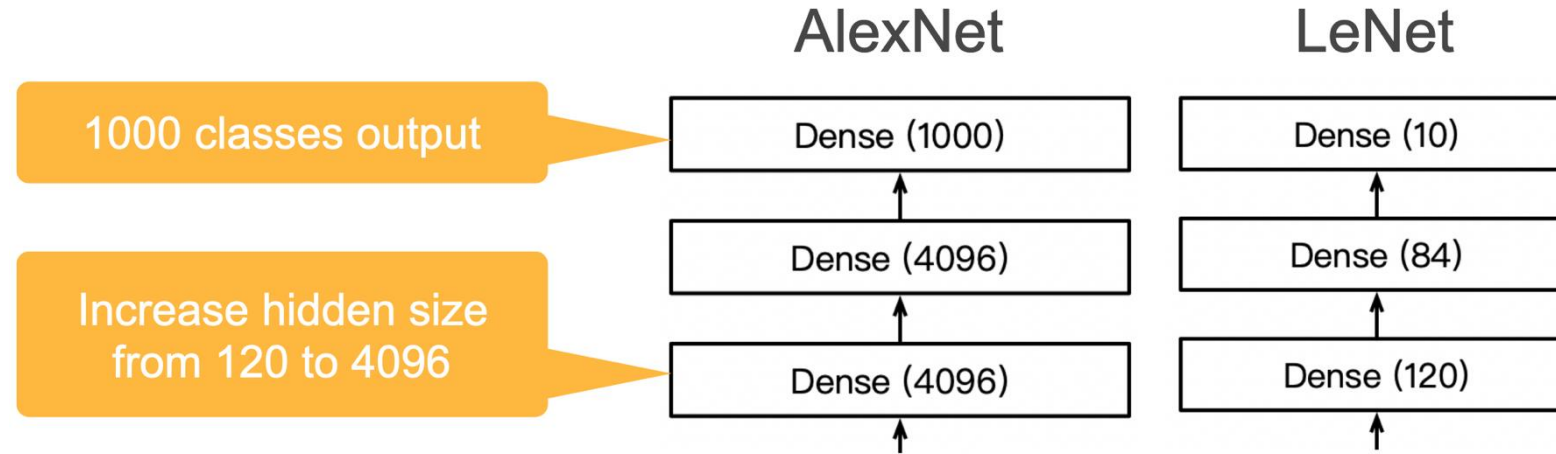
AlexNet



AlexNet



AlexNet

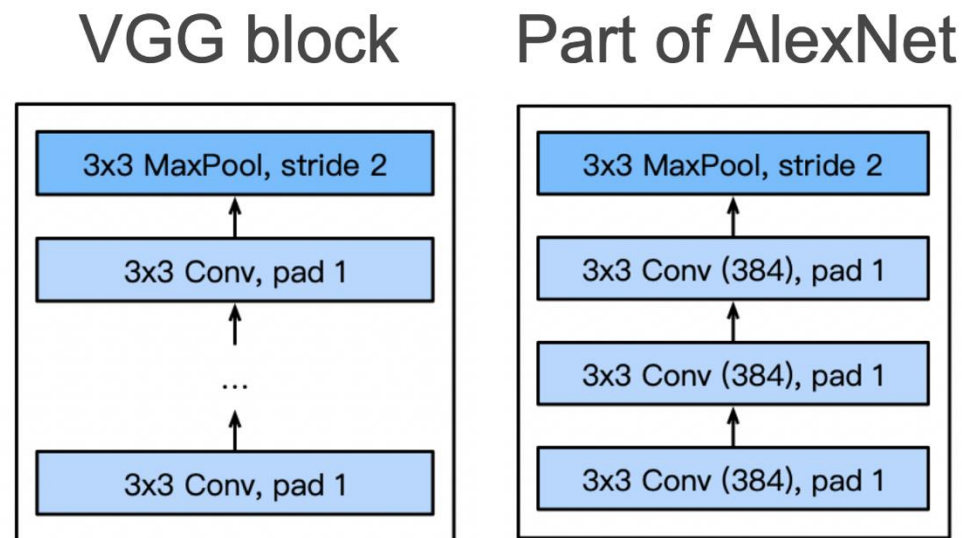


More tricks:

- Change activation function from sigmoid to ReLU (reduce the vanishing gradient problem)
- Add a dropout layer after two hidden dense layers (better robustness / regularization)
- Data augmentation
- GPUs

VGG

- Deeper vs. wider?
 - 7x7 convolutions
 - 3x3 convolutions (more)
 - **Deep & narrow better**
- VGG block
 - 3x3 convolutions (pad 1) (n layers, m channels)
 - 2x2 max-pooling (stride 2)
- 138M parameters



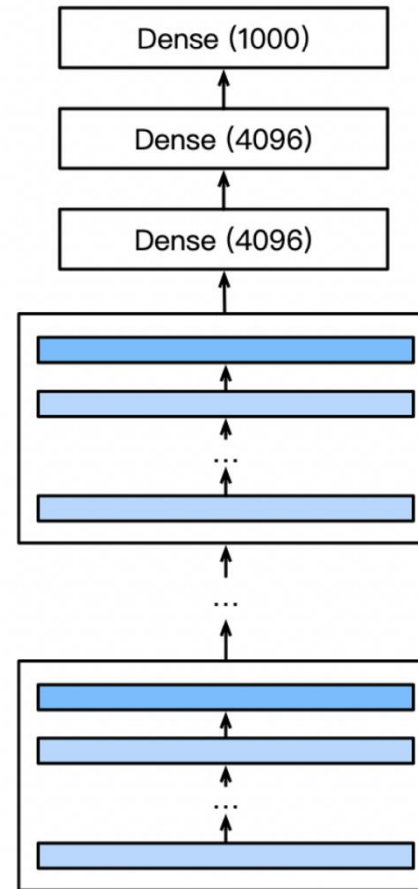
Note:

- 2 stacked layers of 3x3 cover an area of 5x5
- 3 stacked layers of 3x3 cover an area of 7x7
- Number of parameters is reduced

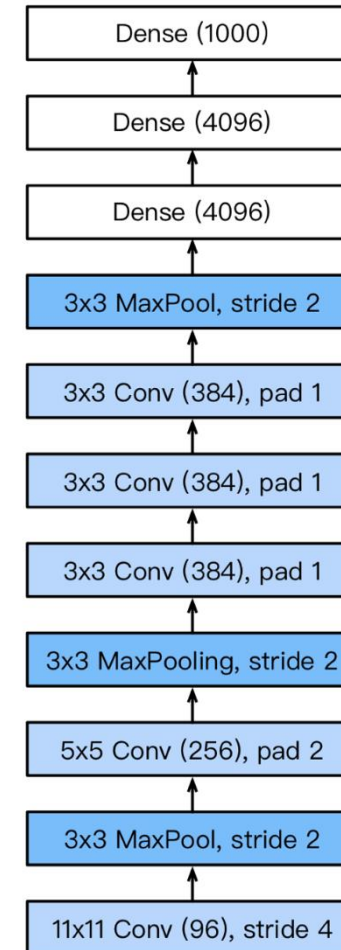
VGG

- Multiple VGG blocks followed by dense layers
 - Increasing number of filters 64-128-256-512
 - Decreasing image size 224-112-56-28-14-7
- Vary the repeating number to get different architectures, such as VGG-16, VGG-19, ...

VGG

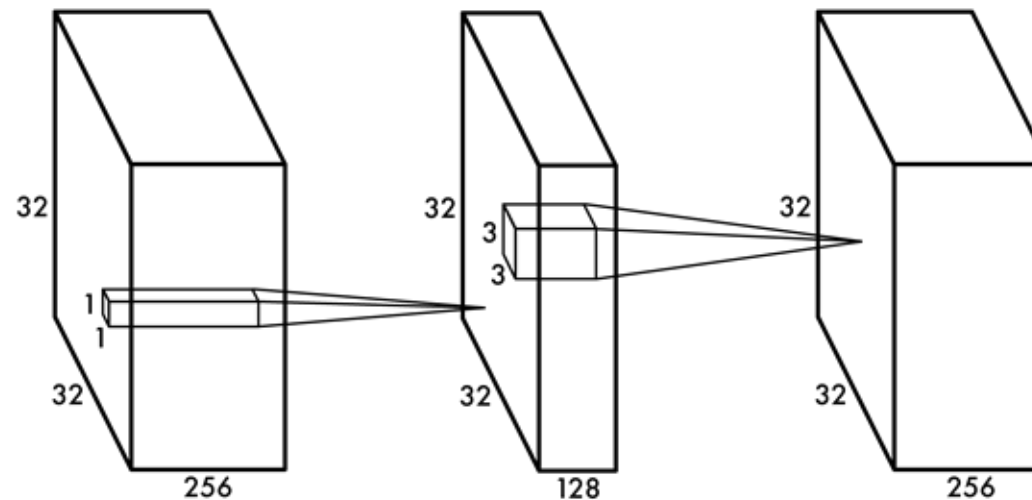


AlexNet



VGG

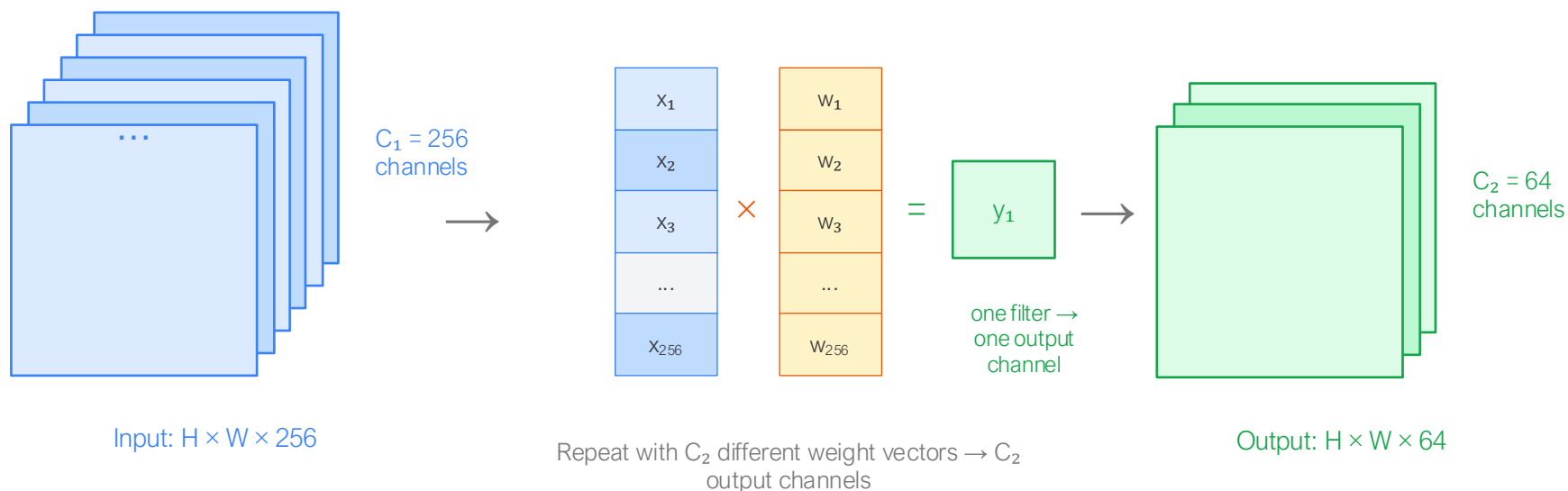
- Just stringing together a bunch of 3x3 convolutions in general, is pretty inefficient.
- Network in network (NiN) idea from Lin et al.:
 - Use 3x3 convolutions to process spatial information
 - Use 1x1 convolutions to reduce # of channels



1×1: Channel-wise transformation

What does a 1×1 convolution do?

A learned linear combination across channels at each spatial location → a fully connected layer applied pixel-by-pixel



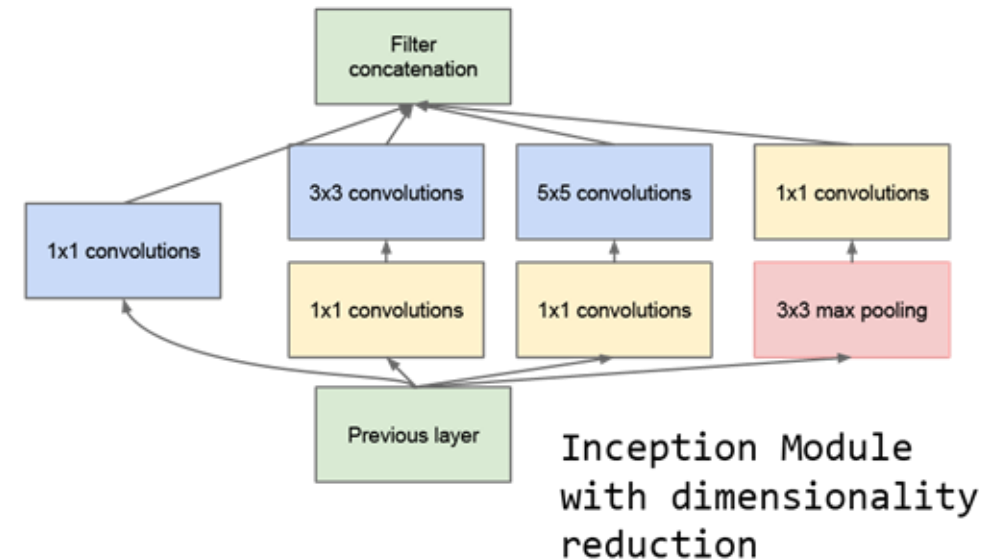
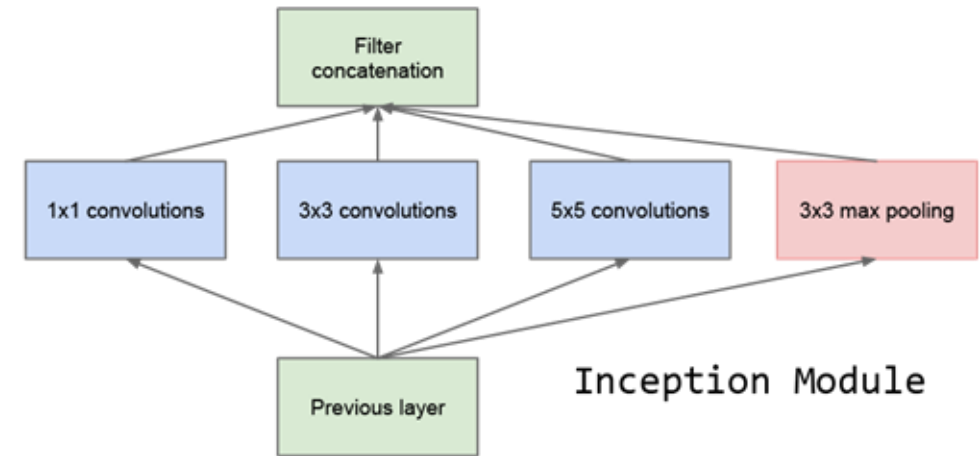
Key uses:

- Channel mixing (Network in Network)
- Dimensionality reduction (Inception bottleneck)
- Bottleneck layers (ResNet-50+)
- Pointwise step in depthwise separable convs (MobileNet)

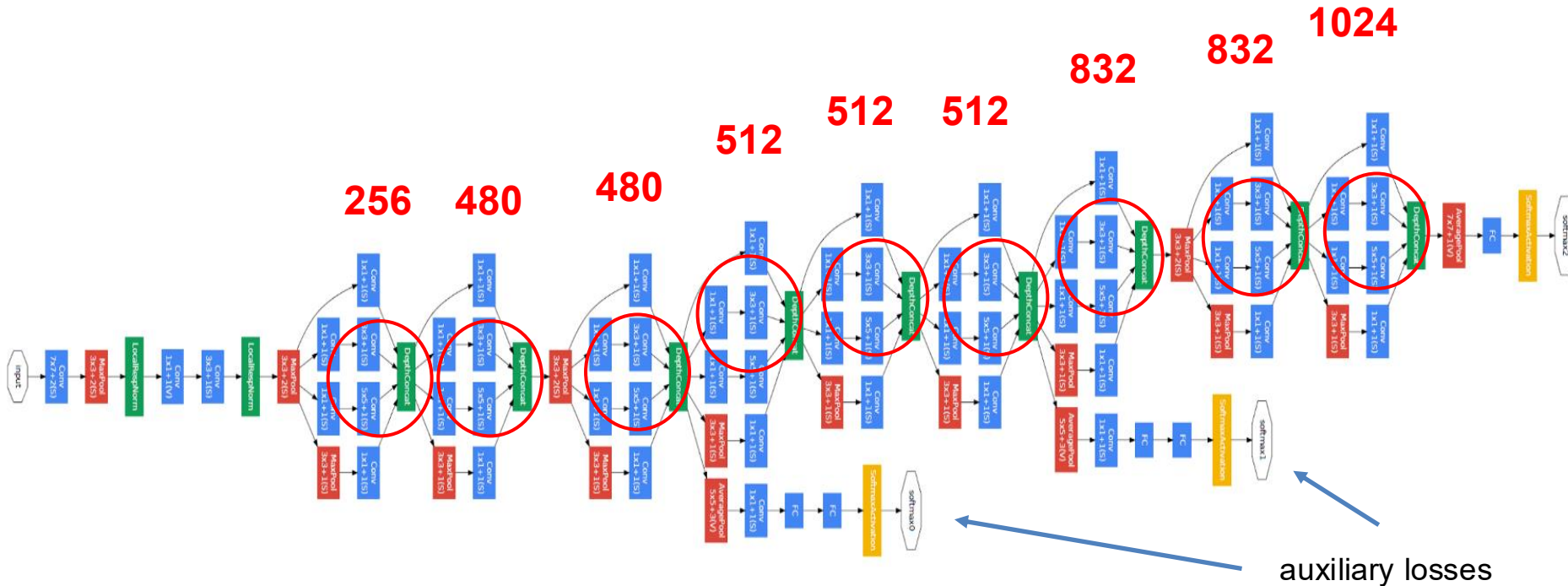
Same spatial size ($H \times W$) — channels reduced from 256 → 64 via learned weighted sums

Inception

- GoogLeNet Inception module → **multi-scale** convolutional module
- Another way to **efficiency**:
 - Split layers
 - Not just one size, but many
- Also use 1x1 convolutions to compress feature maps
- Objective of reducing computational cost but preserving accuracy



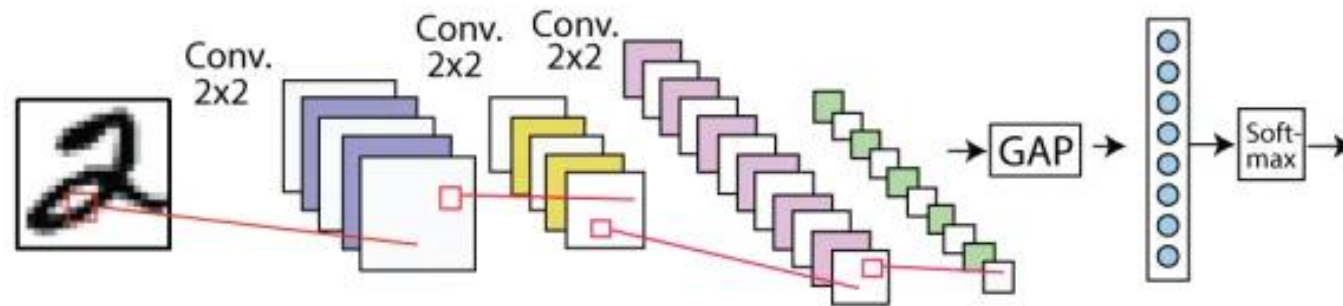
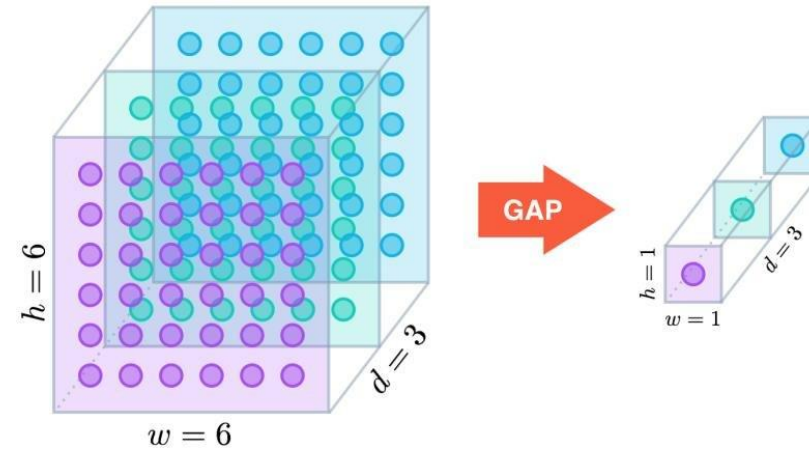
Inception



- Width of inception modules ranges from 256 filters (in early modules) to 1024 in top inception modules.
- Number of parameters: 5 million (much less than VGG)
- There were successive versions of Inception (v1, v2, v3 and v4) looking for more efficiency
- Output uses **Global Average Pooling**

Global Average Pooling (GAP)

- Introduced in the Network in network (NiN) paper
- Typically used as one of the last layers
- Reduces the need for (or totally replaces) the FC layers



The Degradation Problem

Why not just stack more layers?

Counterintuitively, deeper plain networks perform **worse** than shallower ones.

If a shallower model achieves a given accuracy, a deeper model could in theory just copy those layers and add identity mappings, but plain networks can't learn it.

The degradation problem:

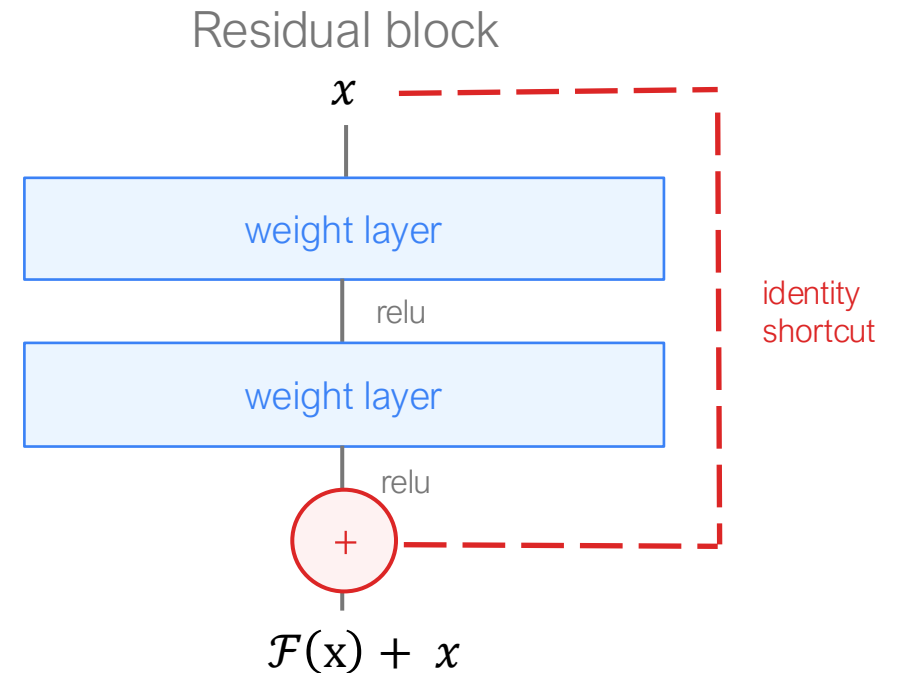
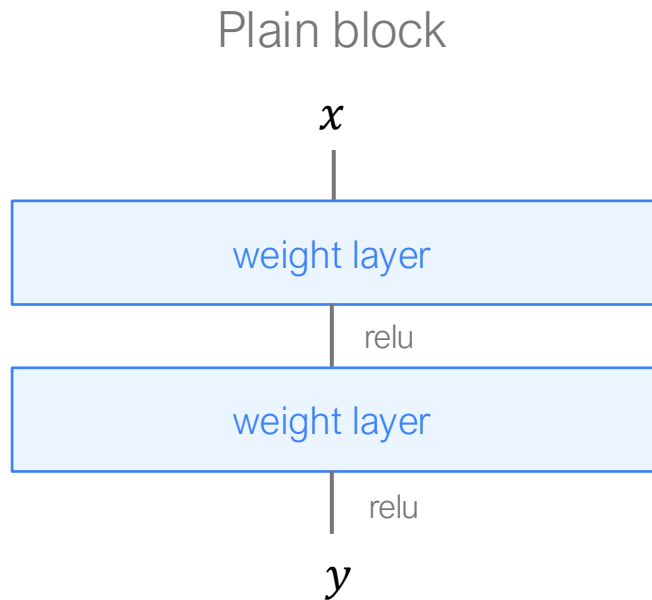
as depth increases, accuracy saturates and then degrades rapidly.



Residual Learning

Key insight: instead of learning $x \rightarrow y$ directly, learn the residual $\mathcal{F}(x) = y - x$

If the identity mapping is optimal, it's easier for the network to push $\mathcal{F}(x) \rightarrow \mathbf{0}$ than to learn $y = x$ from scratch.

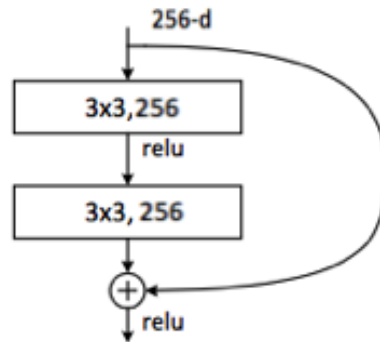


Why it works: Gradients flow directly through the skip connection during backpropagation, preventing vanishing gradients even in very deep networks (100+ layers).

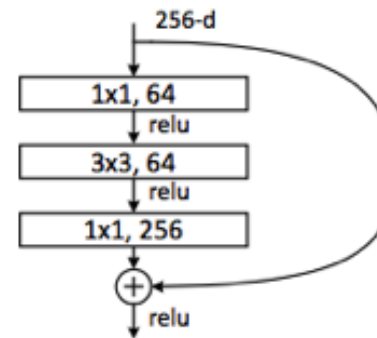
ResNet

- Another version to ResNet made each residual module more efficient with Inception ideas of decomposing convolutions

ResNet 34
residual block



ResNet 50
residual block



- # parameters:
 - ResNet 34 – 21.3M
 - ResNet 50 – 23.5M

Grouped convolutions

- Split up input feature map into groups
- Run convolutions on groups independently
- Recombine

Example:

- 3x3 conv layer 32 x 32 x 256 input, **128 filters, 32 groups**
 - Split input into **32 different feature maps**
 - Each is 32 x 32 x 8
 - Run 4 filters, 3x3x8 on each group
 - Merge 4*32 channels back together, get 32 x 32 x 128 output

Input, output stays same dimensions, less computation

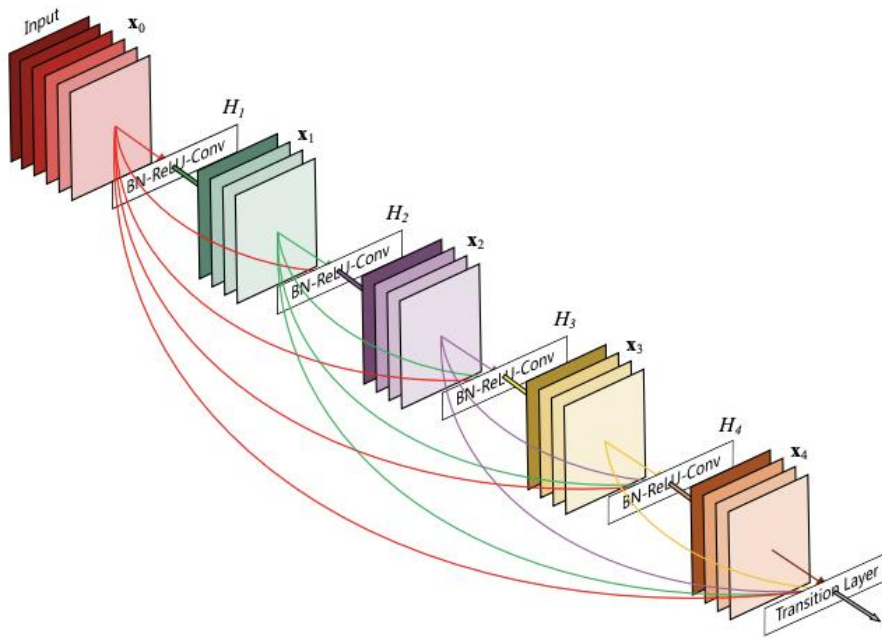
ResNeXt

- Replace 3x3 blocks with larger grouped convs
- “Larger” network but same computational complexity

stage	output	ResNet-50	ResNeXt-50 (32×4d)
conv1	112×112	7×7, 64, stride 2	7×7, 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128, C=32 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256, C=32 \\ 1\times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512, C=32 \\ 1\times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 1024 \\ 3\times 3, 1024, C=32 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		25.5×10^6	25.0×10^6
FLOPs		4.1×10^9	4.2×10^9

DenseNet

- DenseNet is an extension to ResNet
- Instead of adding the input to the mapping function, DenseNet **concatenates** them, so any previous learned features can be **reused** later

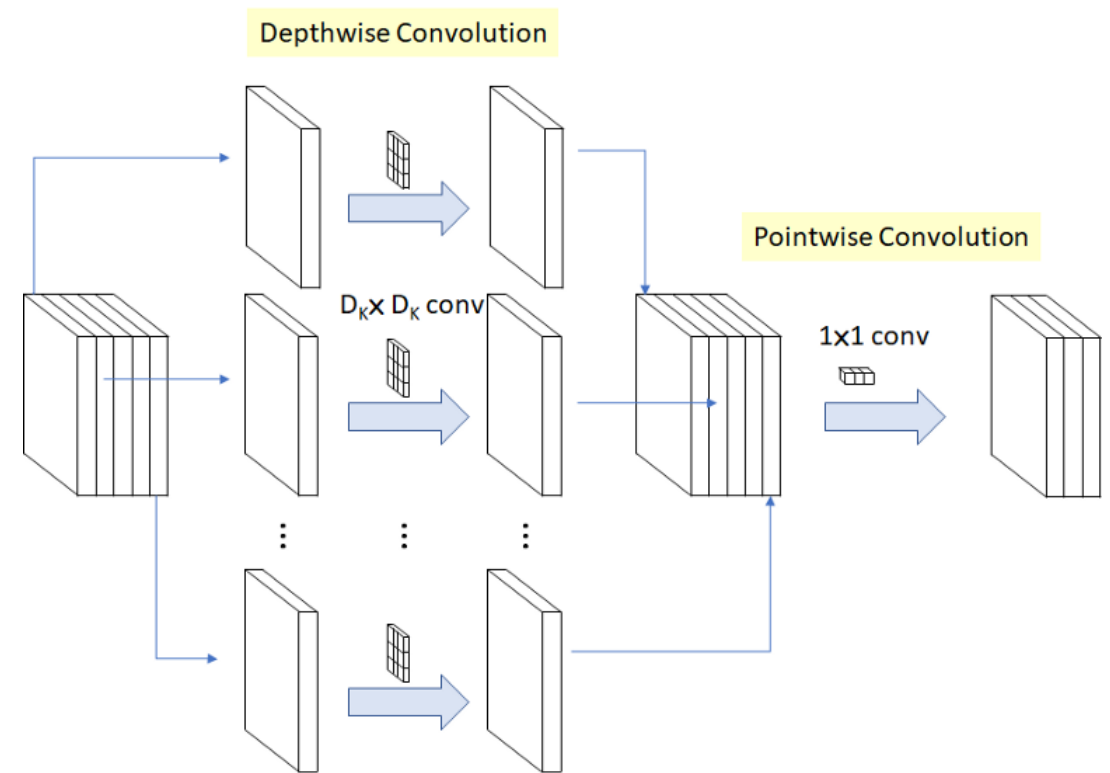


Advantages over ResNet:

- Feature reuse → earlier features are directly available to later layers
- Fewer new channels needed per layer (growth rate $k = 12-32$)
- Better parameter efficiency — DenseNet-201 outperforms ResNet-101 with fewer parameters
- Stronger gradient flow — implicit deep supervision

MobileNet

- These models can be **computationally expensive** → How can real-time performance be achieved using mobile or other embedded devices?
- MobileNet designed for memory and computation restrictions
 - Depth-separable convolutions
 - Reduce the number of parameters and computation
- MobileNetV2 introduced tweaks like residual connections to reduce even more the parameters



CNN Architectures: Trade-offs

Depth vs. Trainability

Deeper networks should learn richer representations, but vanishing gradients and degradation make them hard to train.

Accuracy vs. Efficiency

Higher accuracy often requires more parameters and FLOPs.

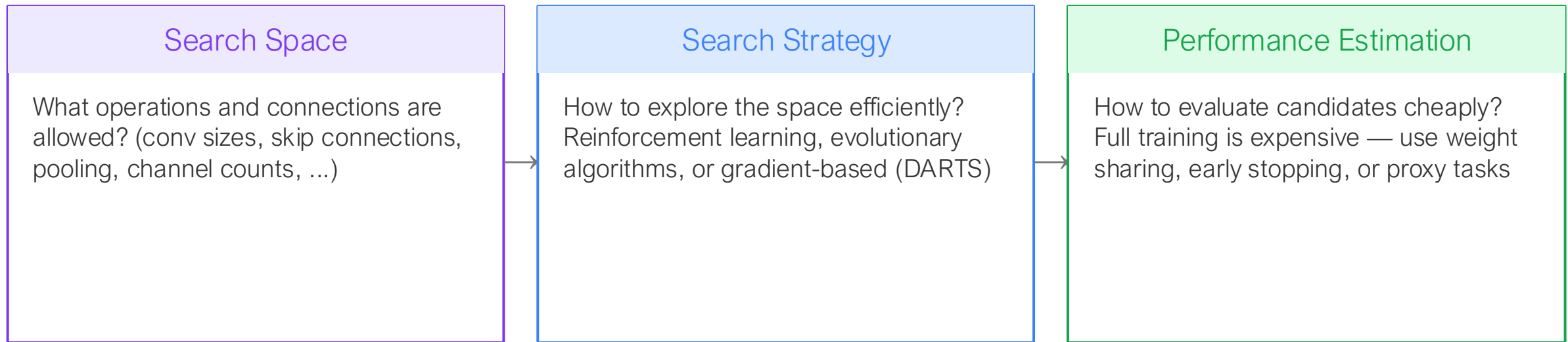
Manual vs. Automated Design

Every architecture so far was hand-designed. Can machines find better ones?

Neural Architecture Search (NAS)

Can machines design better architectures than humans?

Every architecture so far (AlexNet, VGG, Inception, ResNet, ...) was hand-designed → NAS automates this process



NASNet (Zoph et al., 2018) and MnasNet (Tan et al., 2019) outperformed hand-designed architectures on ImageNet, proving that automated search can surpass human intuition.

EfficientNet: Compound Scaling

How should we scale a CNN to get better accuracy?

Prior work scaled one dimension at a time. EfficientNet shows that **balancing all three** yields much better results.

↕ Depth (d)

More layers → richer features, but diminishing returns and harder to train

↔ Width (w)

More channels per layer → finer-grained features, but quickly saturates

▣ Resolution (r)

Larger input images → more detail, but quadratic increase in computation

Starting from a NAS-derived baseline (EfficientNet-B0), compound scaling produces a family of architectures B0–B7

EfficientNet-B7: 84.3% top-1 ImageNet accuracy — 8.4× smaller and 6.1× faster than the best prior CNN (GPipe)

CNN Architectures: Where next?

2017–2019

Efficient Architectures

MobileNetV2/V3, ShuffleNet, EfficientNet — focus shifts to accuracy per FLOP, not just raw accuracy.

2019–2020

Self-Supervised Pretraining

SimCLR, MoCo, BYOL — learning representations without labels, reducing dependence on ImageNet supervision.

2020–2021

Vision Transformers (ViT)

Attention replaces convolution: image patches as tokens. Requires large-scale data or strong augmentation.

2022+

Hybrid & Modern CNNs

ConvNeXt shows that modernizing CNNs (with Transformer-era tricks) makes them competitive with ViTs again.